

LAPORAN PRATIKUM PEKAN 9
STRUKTUR DATA
SEARCHING (BFS dan DFS)



Disusun oleh:
Jose Moniz de Araujo 2511538002

Dosen pengampu:
Wahyudi Dr. S.T.M.T

Asisten Pratikum:
Sherly Sukmadira

DEPARTAMENT INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN
2026

KATA PENGATAR

Saya ingin mengucapkan rasa syukur kepada Tuhan Yang Maha Esa atas berkat-Nya, saya dapat menyelesaikan laporan praktikum ini dengan baik. Dokumen ini dibuat untuk memenuhi tugas praktik pada mata kuliah pemrograman terkait topik sorting yang meliputi insertion sort, selection sort, dan bubble sort.

Dalam penyusunan laporan ini, saya mengetahui bagaimana cara proses pengurutan data dilakukan dengan beberapa metode sorting dasar serta memahami cara kerja masing-masing algoritma dalam bahasa pemrograman java.

Saya sadar bahwa laporan ini masih ada beberapa kekurangan, baik dari segi penulisan maupun dari isi materi. Karena itu, saya mengharapkan umpan dan saran yang membangun supaya laporan ini bisa ditingkatkan di kemudian hari.

Saya juga mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah memberikan kontribusi dalam pembuatan laporan praktikum ini. Kami berharap laporan ini bermanfaat bagi pembaca dan dapat memperdalam pemahaman tentang algoritma pengelolaan data.

Padang, 13 Mei 2026

Jose Moniz de Araujo

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data sangat penting dalam ilmu komputer untuk mengelola dan mengatur semua data agar pencarian lebih efisien. Salah satu aplikasi umum dari struktur data adalah graf, yang merupakan kumpulan simpul yang biasanya dihubungkan oleh sisi.

Dalam pemrosesan graf, terdapat berbagai teknik pencarian atau inferensi. Dua algoritma pencarian yang paling umum digunakan adalah Breadth-First Search (BFS) dan Depth-First Search (DFS). BFS mencari dengan menjelajahi semua simpul pada tingkat yang sama sebelum berpindah ke tingkat berikutnya, sedangkan DFS mencari simpul hingga kedalaman tertentu sebelum kembali dan menjelajahi jalur lain.

Algoritma BFS dan DFS memainkan peran penting dalam banyak aplikasi komputer, termasuk pencarian jalur, analisis jaringan komputer, kecerdasan buatan, media sosial, dan pemetaan rute. Oleh karena itu, memahami konsep dan aplikasi

1.2 Tujuan praktikum

1. Memahami dasar pada konsep algoritma Breadth-First search (BFS) dan (DFS).
2. Mengetahui perbedaan cara kerja BFS dan DFS.
3. Mengimplementasikan algoritma BFS dan DFS dalam program.

1.3 manfaat Praktikum

1. Menambah pemahaman tentang struktur data Graf
2. Meningkatkan kemampuan dalam menerapkan algoritma pencarian pada graf.
3. Melatih kemampuan pemrograman dalam mengimplementasikan BFS dan DFS.

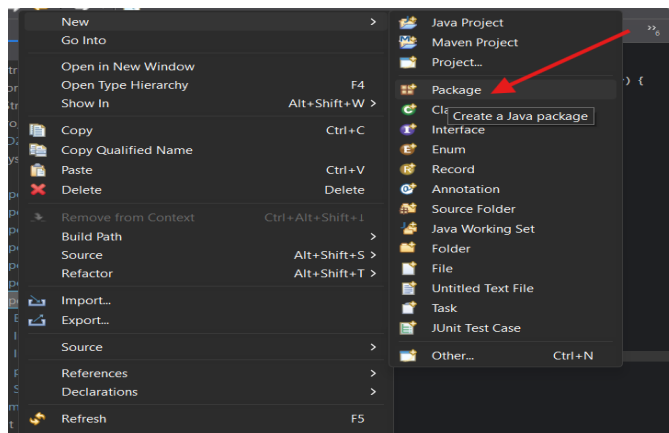
BAB II

PEMBAHASAN

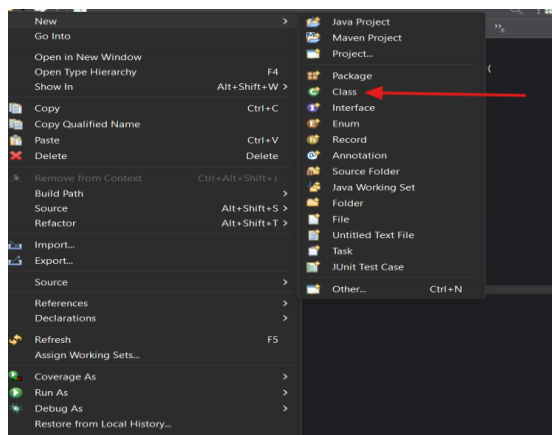
2.1 2.1 Kode Program praktikum pekan 9

2.1.1 Node_2511538002

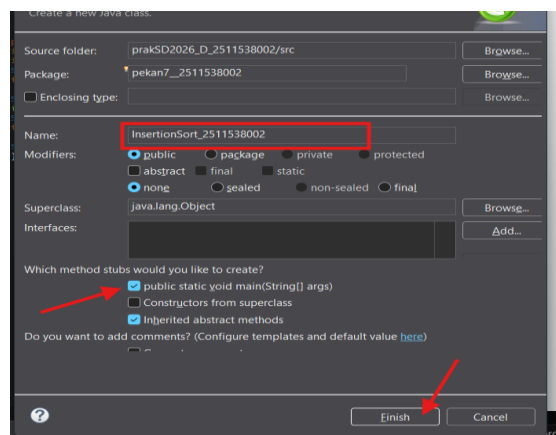
1. Buka aplikasi Eclipse lalu buat package baru pada src klik kanan, lalu pilih “New” dan klik tulisan package.



2. Kemudian buat nama packagenya tanpa spasi, huruf kapital, karakter khusus. Beri nama “pekan9_2511538002”. Lalu klik finish.
3. Ketika package sudah jadi, klik pada bagian pekan9_2511538002”. Lalu klik “New” dan pilih bagian “class” untuk membuat program.



4. Buat nama program yang akan dibuat pada bagian nama tanpa spasi dan menggunakan huruf kapital pada awal kata, lalu klik bagian “public static void main(String[] args)”. Kemudian klik “Finish”



5. Masukan Syntax berikut:

```
1 package pekan9_2511538002;
2
3 public class Node_2511538002 {
4     int data_8002;
5     Node_2511538002 left_8002;
6     Node_2511538002 right_8002;
7     public Node_2511538002(int data_8002) {
8         this.data_8002 = data_8002;
9         left_8002 = null;
10        right_8002 = null;
11    }
12
13    public void setLeft_8002(Node_2511538002 node_8002) {
14        if (left_8002 == null)
15            left_8002 = node_8002;
16    }
17
18    public void setRight_8002(Node_2511538002 node_8002) {
19        if (right_8002 == null)
20            right_8002 = node_8002;
21    }
22
23    public Node_2511538002 getLeft_8002() {
24        return left_8002;
25    }
26
27    public Node_2511538002 getRight_8002() {
28        return right_8002;
29    }
30
31    public int getData() {
32        return data_8002;
33    }
34
35    public void setData_8002(int data_8002) {
36        this.data_8002 = data_8002;
37    }
38
39    void PrintPreorder_8002(Node_2511538002 node_8002) {
40        if (node_8002 == null)
41            return;
42        System.out.print(node_8002.data_8002 + " ");
43        printPreorder_8002(node_8002.left_8002);
44        printPreorder_8002(node_8002.right_8002);
45    }
46
47    void printPreorder_8002(Node_2511538002 node_8002) {
48        if (node_8002 == null)
49            return;
50        printPostorder(node_8002.left_8002);
51        printPostorder(node_8002.right_8002);
52        System.out.print(node_8002.data_8002 + " ");
53    }
54 }
```

```

48 void printInorder_8002(Node_2511538002 node_8002) {
49     if (node_8002 == null) {
50         return;
51     }
52     printInorder_8002(node_8002.left_8002);
53     System.out.print(node_8002.data_8002 + " ");
54     printInorder_8002(node_8002.right_8002);
55 }
56 void printPostorder_8002(Node_2511538002 node_8002) {
57     if (node_8002 == null) {
58         return;
59     }
60     printPostorder_8002(node_8002.left_8002);
61     printPostorder_8002(node_8002.right_8002);
62     System.out.print(node_8002.data_8002 + " ");
63 }
64 void printInorder_8002(Node_2511538002 node_8002) {
65     if (node_8002 == null) {
66         return;
67     }
68     printInorder_8002(node_8002.left_8002);
69     System.out.print(node_8002.data_8002 + " ");
70     printInorder_8002(node_8002.right_8002);
71 }
72 public String print() {
73     return this.print("", true, "");
74 }
75 public String print(String prefix, boolean isTail, String sb) {
76     if (right_8002 != null) {
77         right_8002.print(prefix + (isTail ? " : " : " "), false, sb);
78     }
79     System.out.println(prefix + (isTail ? " : " : " ") + data_8002);
80     if (left_8002 != null) {
81         left_8002.print(prefix + (isTail ? " : " : " "), true, sb);
82     }
83     return sb;
84 }
85 }
86 public static void main(String[] args) {
87     // TODO Auto-generated method stub
88 }
89 }
90 }
91 }
92 }

```

2.1.2 BTree_2511538002

- 1 Ulangi pembuatan class seperti program sebelumnya dan beri nama class tersebut “BTree_2511538002”. Lalu masukan Syntax seperti berikut:

```

1 package pekany_2511538002;
2
3 public class BTree_2511538002 {
4     private Node_2511538002 root;
5     private Node_2511538002 currentNode;
6     public BTree_2511538002() {
7         root = null;
8     }
9
10    public boolean search(int data_8002) {
11        return search(root, data_8002);
12    }
13
14    private boolean search(Node_2511538002 node_8002, int data_8002) {
15        if (node_8002.getData() == data_8002) {
16            return true;
17        }
18        if (node_8002.getLeft_8002() != null) {
19            if (search(node_8002.getLeft_8002(), data_8002)) {
20                return true;
21            }
22        }
23        if (node_8002.getRight_8002() != null) {
24            if (search(node_8002.getRight_8002(), data_8002)) {
25                return true;
26            }
27        }
28        return false;
29    }
30
31    public void printInorder_8002() {
32        root.printInorder_8002(root);
33    }
34
35    public void printpreOrder_8002() {
36        root.printPreorder_8002(root);
37    }
38
39    public void printpostOrder_8002() {
40        root.printPostorder_8002(root);
41    }
42
43    public Node_2511538002 getRoot() {
44        return root;
45    }
46
47    public boolean isEmpty_8002() {
48        return root == null;
49    }
50
51    public int countNodes() {
52        return countNodes(root);
53    }
54
55    private int countNodes(Node_2511538002 node_8002) {
56        int count = 1;
57        if (node_8002 == null) {
58            return 0;
59        }
60        else {
61            count += countNodes(node_8002.getLeft_8002());
62            count += countNodes(node_8002.getRight_8002());
63            return count;
64        }
65    }
66 }
67 }
68 }
69 }
70 }
71 }
72 }

```

```

53     public void print() {
54         root.print();
55     }
56     public Node_2511538002 getCurrent() {
57         return currentNode;
58     }
59     public void setCurrent(Node_2511538002 node_8002) {
60         this.currentNode = node_8002;
61     }
62     public void setRoot(Node_2511538002 root) {
63         this.root = root;
64     }
65     public static void main(String[] args) {
66         // TODO Auto-generated method stub
67     }
68 }
69
70 }
71

```

2.1.3 BtreeDriver_2511538002

1. Ulangi pembuatan class seperti program sebelumnya dan beri nama class tersebut “BtreeDriver_2511538002”. Lalu masukan Syntax seperti berikut:

```

1 package pekan9_2511538002;
2
3
4 public class BtreeDriver_2511538002 {
5
6     public static void main(String[] args) {
7         //membuat pohon
8         BTree_2511538002 tree = new BTree_2511538002();
9         System.out.println("Jumlah simpul awal pohon: ");
10        System.out.println(tree.countNodes());
11        // menambahkan simpul data 1
12        Node_2511538002 root = new Node_2511538002 (1);
13        // menjadikan simpul 1 sebagai root
14        tree.setRoot(root);
15        System.out.println("jumlah simpul jika hanya ada root");
16        System.out.println(tree.countNodes());
17        Node_2511538002 node2 = new Node_2511538002(2);
18        Node_2511538002 node3 = new Node_2511538002(3);
19        Node_2511538002 node4 = new Node_2511538002(4);
20        Node_2511538002 node5 = new Node_2511538002(5);
21        Node_2511538002 node6 = new Node_2511538002(6);
22        Node_2511538002 node7 = new Node_2511538002(7);
23        Node_2511538002 node8 = new Node_2511538002(8);
24        Node_2511538002 node9 = new Node_2511538002(9);
25        root.setLeft_8002(node2);
26        node2.setLeft_8002(node4);
27        node2.setRight_8002(node5);
28        node4.setRight_8002(node8);
29        root.setRight_8002(node3);
30        node3.setLeft_8002(node6);
31        node3.setRight_8002(node7);
32        node6.setLeft_8002(node9);
33        // set root
34        tree.setCurrent(tree.getRoot());
35        System.out.println("menampilkan simpul terakhir: ");
36        System.out.println(tree.getCurrent().getData());
37        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
38        System.out.println(tree.countNodes());
39        System.out.println("InOrder: ");
40        tree.printInorder_8002();
41        System.out.println("\npostorder : ");
42        tree.printpostOrder_8002();
43        System.out.println("\n Dimenampilkan simpul dalam bentuk pohon");
44        tree.print();
45        // TODO Auto-generated method stub
46
47    }
48 }
49
50

```

2. Kemudian klik Run untuk melihat hasil dari Syntax yang suda dibuat:

```
<terminated> BtreeDriver_2511538002 [Java Application] C:\
Jumlah simpul awal pohon: 0
jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
postorder :
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
|-- 7
|
|-- 3
|   |-- 6
|   |-- 9
|
|-- 1
|   |-- 5
|   |-- 2
|       |-- 8
|       |-- 4
```

2.1.4 GraphTraversal_2511538002

1. Ulangi pembuatan class seperti program sebelumnya dan beri nama class tersebut “GraphTraversal_2511538002”. Lalu masukan Syntax seperti berikut:

```
1 package pekan9_2511538002;
2 import java.util.*;
3
4 public class GraphTraversal_2511538002 {
5     private Map<String, List<String>> graph = new HashMap<>();
6     //menambahkan edge
7     public void addEdge (String node1, String node2) {
8         graph.putIfAbsent(node1, new ArrayList<>());
9         graph.putIfAbsent(node2, new ArrayList<>());
10        graph.get(node1).add(node2);
11        graph.get(node2).add(node1);
12    }
13    // menampilkan graf awal
14    public void printGraph() {
15        System.out.println("graf awal (adjacency list):");
16        for (String node_8002 : graph.keySet()) {
17            System.out.print(node_8002 + "->");
18            List<String> neighbors = graph.get(node_8002);
19            System.out.println(String.join(", ", neighbors));
20        }
21        System.out.println();
22    }
23    //DFS rekursif
24    public void dfs(String Start) {
25        Set<String> visited = new HashSet<>();
26        System.out.println("penelusuran DFS: ");
27        dfsHelper(Start, visited);
28        System.out.println();
29    }
30    private void dfsHelper (String current, Set<String> visited) {
31        if (visited.contains(current)) return;
32        visited.add(current);
33        System.out.print(current + " ");
34        for (String neighbor : graph.getOrDefault(current, new ArrayList<>())) {
35            dfsHelper(neighbor, visited);
36        }
37    }
38 }
```



```

38
39 // bfs public iteratif
40 public void bfs(String start) {
41     Set<String> visited = new HashSet<>();
42     Queue<String> queue = new LinkedList<>();
43     queue.add(start);
44     visited.add(start);
45     System.out.println("Penelusuran BFS: ");
46     while (!queue.isEmpty()) {
47         String current = queue.poll();
48         System.out.print(current + " ");
49         for (String neighbor : graph.getDefault(current, new ArrayList<>())) {
50             if (!visited.contains(neighbor)) {
51                 queue.add(neighbor);
52                 visited.add(neighbor);
53             }
54         }
55     }
56     System.out.println();
57 }
58 // main
59 public static void main(String[] args) {
60     GraphTraversal_2511538002 graph = new GraphTraversal_2511538002();
61     // contoh.addEdge(A-B....
62     graph.addEdge("A", "B");
63     graph.addEdge("A", "C");
64     graph.addEdge("B", "D");
65     graph.addEdge("B", "E");
66     // contoh.printGraph();
67     System.out.println("Graf awal adalah: ");
68     graph.printGraph();
69
70     // lakukan penelusuran
71     graph.dfs("A");
72     graph.bfs("A");
73 }
74 }
75

```

2. Kemudian klik Run untuk melihat hasil dari Syntax yang suda dibuat:

```

<terminated> GraphTraversal_2511538002 [Java Ap
Graf awal adalah:
graf awal (adjacency list):
A->B, C
B->A, D, E
C->A
D->B
E->B

penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E

```

2.2. Tugas Praktikum Pekan_9

2.2.1 Kode Program Mahasiswa_2511538002

```
1 package pekan9_2511538002;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.util.*;
6 import java.util.List;
7
8 public class PetaKampus_2511538002 extends JFrame {
9
10     // Struktur Graph menggunakan Adjacency List
11     private Map<String, List<String>> graph_8002 = new LinkedHashMap<>();
12     private Map<String, Point> positions = new HashMap<>();
13
14     // Penampung Hasil Pencarian
15     private List<String> visitedNodes = new ArrayList<>();
16     private List<String> finalPath = new ArrayList<>();
17     private int exploredCount = 0;
18
19     // Komponen GUI
20     private JComboBox<String> startCombo;
21     private JComboBox<String> goalCombo;
22     private JTextArea resultArea;
23     private GraphPanel graphPanel;
24
25     public PetaKampus_2511538002() {
26         setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
27         setSize(650, 480);
28         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29         setLocationRelativeTo(null);
30         setLayout(new BorderLayout());
31
32         // Inisialisasi 7 Node & Sisi sesuai gambac
33         initGraphData();
34
35         // 1. Panel Atas (Kontrol & Tombol Utama)
36         JPanel topPanel = new JPanel(new GridBagLayout());
37         topPanel.setBackground(new Color(238, 238, 238));
38         GridBagConstraints gbc = new GridBagConstraints();
39         gbc.insets = new Insets(8, 8, 8, 8);
40         gbc.fill = GridBagConstraints.HORIZONTAL;
41
42         String[] locations = graph_8002.keySet().toArray(new String[0]);
43         startCombo = new JComboBox<>(locations);
44         goalCombo = new JComboBox<>(locations);
45
46         // Atur default combo box sesuai gambac
47         startCombo.setSelectedItem("Rektorat");
48         goalCombo.setSelectedItem("Labor AI");
49
50         // Basis 1: Lokasi Awal & Tombol
51         gbc.gridx = 0; gbc.gridy = 0;
52         topPanel.add(new JLabel("Lokasi Awal : "), gbc);
53         gbc.gridx = 1;
54         topPanel.add(startCombo, gbc);
55
56         gbc.gridx = 2;
57         JButton btnBFS = new JButton("[ BFS ]");
58         btnBFS.setBackground(new Color(144, 238, 144)); // Hijau Iscane
59         topPanel.add(btnBFS, gbc);
60
61         gbc.gridx = 3;
62         JButton btnDFS = new JButton("[ DFS ]");
63         btnDFS.setBackground(new Color(255, 182, 73)); // Oranye
64         topPanel.add(btnDFS, gbc);
65
66         gbc.gridx = 4;
67         JButton btnReset = new JButton("[ RESET ]");
68         btnReset.setBackground(new Color(255, 120, 120)); // Merah muda/salmon
69         topPanel.add(btnReset, gbc);
70
71         // Basis 2: Lokasi Tujuan
72         gbc.gridx = 0; gbc.gridy = 1;
73         topPanel.add(new JLabel("Lokasi Tujuan : "), gbc);
74         gbc.gridx = 1;
75         topPanel.add(goalCombo, gbc);
76
77         add(topPanel, BorderLayout.NORTH);
78
79         // 2. Panel Tengah (Visualisasi Graph sesuai gambac)
80         graphPanel = new GraphPanel();
81         add(graphPanel, BorderLayout.CENTER);
82
83         // 3. Panel Bawah (Hasil Pencarian Laka)
84         JPanel bottomPanel = new JPanel(new BorderLayout());
85         bottomPanel.setBackground(Color.WHITE);
86
87         resultArea = new JTextArea(5, 40);
88         resultArea.setEditable(false);
89         resultArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
90         resultArea.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15));
91         bottomPanel.add(resultArea, BorderLayout.CENTER);
92
93         add(bottomPanel, BorderLayout.SOUTH);
94
95         // Event Listener Aksi Tombol
96         btnBFS.addActionListener(e -> BFS());
97         btnDFS.addActionListener(e -> DFS());
98         btnReset.addActionListener(e -> resetGraph());
99
100         resetGraph(); // Set text format pertama kali
101     }
102
103     private void initGraphData() {
104         // Grafik 7 Node Utama sesuai gambac
105         String[] nodes = {"Perpus", "FIT", "Rektorat", "PKM", "Lab Jaringan", "Masjid", "Labor AI"};
106         for (String node : nodes) {
107             graph_8002.put(node, new ArrayList<>());
108         }
109     }
110 }
```

```

110 // Koordinat Layout Grid Icha (X, Y) agar posisi sesuai
111 positions.put("Perpus", new Point(180, 50));
112 positions.put("FTI", new Point(420, 50));
113
114 positions.put("Rektorat", new Point(180, 120));
115 positions.put("PKM", new Point(380, 120));
116 positions.put("Lab Jaringan", new Point(420, 120));
117
118 positions.put("Masjid", new Point(180, 190));
119 positions.put("Labor AI", new Point(420, 190));
120
121 // Struktur Sisi Relasi Garis (Undirected)
122 addEdge("Perpus", "FTI");
123 addEdge("Perpus", "Rektorat");
124 addEdge("Rektorat", "PKM");
125 addEdge("PKM", "Lab Jaringan");
126 addEdge("FTI", "Lab Jaringan");
127 addEdge("Rektorat", "Masjid");
128 addEdge("Masjid", "Labor AI");
129 addEdge("Lab Jaringan", "Labor AI");
130 }
131
132 private void addEdge(String source, String dest) {
133     graph_8002.get(source).add(dest);
134     graph_8002.get(dest).add(source);
135 }
136
137 public void BFS() {
138     String start = (String) startCombo.getSelectedItem();
139     String goal = (String) goalCombo.getSelectedItem();
140
141     visitedNodes.clear();
142     finalPath.clear();
143     exploredCount = 0;
144
145     Queue<String> queue = new LinkedList<>();
146     Map<String, String> parentMap = new HashMap<>();
147
148     queue.add(start);
149     visitedNodes.add(start);
150
151     boolean found = false;
152
153     while (!queue.isEmpty()) {
154         String current = queue.poll();
155         exploredCount++;
156
157         if (current.equals(goal)) {
158             found = true;
159             break;
160         }
161
162         for (String neighbor : graph_8002.get(current)) {
163             if (!visitedNodes.contains(neighbor)) {

```

```

164                 visitedNodes.add(neighbor);
165                 parentMap.put(neighbor, current);
166                 queue.add(neighbor);
167             }
168         }
169     }
170
171     if (found) {
172         String curr = goal;
173         while (curr != null) {
174             finalPath.add(0, curr);
175             curr = parentMap.get(curr);
176         }
177     }
178
179     displayPath("BFS");
180     graphPanel.repaint();
181 }
182
183 public void DFS() {
184     String start = (String) startCombo.getSelectedItem();
185     String goal = (String) goalCombo.getSelectedItem();
186
187     visitedNodes.clear();
188     finalPath.clear();
189     exploredCount = 0;
190
191     Stack<String> stack = new Stack<>();
192     Map<String, String> parentMap = new HashMap<>();
193     Set<String> visitedSet = new LinkedHashSet<>();
194
195     stack.push(start);
196
197     boolean found = false;
198
199     while (!stack.isEmpty()) {
200         String current = stack.pop();
201
202         if (!visitedSet.contains(current)) {
203             visitedSet.add(current);
204             visitedNodes.add(current);
205             exploredCount++;
206
207             if (current.equals(goal)) {
208                 found = true;
209                 break;
210             }
211
212             List<String> neighbors = graph_8002.get(current);
213             for (int i = neighbors.size() - 1; i >= 0; i--) {
214                 String neighbor = neighbors.get(i);
215                 if (!visitedSet.contains(neighbor)) {
216                     stack.push(neighbor);

```

```

217         parentMap.put(neighbor, current);
218     }
219 }
220 }
221 }
222
223 if (found) {
224     String curr = goal;
225     while (curr != null) {
226         finalPath.add(0, curr);
227         curr = parentMap.get(curr);
228     }
229
230     displayPath("DFS");
231     graphPanel.repaint();
232 }
233
234
235 public void displayPath(String methodType) {
236     StringBuilder sb = new StringBuilder();
237     sb.append("Hasil Pencarian : \n");
238     sb.append("Jalur : ").append(String.join(" -> ", finalPath)).append("\n");
239     sb.append("Node Dikunjungi : ").append(String.join(", ", visitedNodes)).append("\n");
240     sb.append("Jumlah Node Dikunjungi : ").append(exploredCount);
241     resultArea.setText(sb.toString());
242 }
243
244 public void resetGraph() {
245     visitedNodes.clear();
246     finalPath.clear();
247     exploredCount = 0;
248     resultArea.setText("Hasil Pencarian : \nJalur : \nNode Dikunjungi : \nJumlah Node Dikunjungi : 0");
249     graphPanel.repaint();
250 }
251
252 // Panel khusus pengasambakan Grafis tanpa lingkaran Node
253 private class GraphPanel extends JPanel {
254     public GraphPanel() {
255         setBackground(new Color(240, 240, 240));
256         setBorder(BorderFactory.createTitledBorder(
257             BorderFactory.createLineBorder(Color.LIGHT_GRAY), "VISUALISASI GRAPH"));
258     }
259
260     @Override
261     protected void paintComponent(Graphics g) {
262         super.paintComponent(g);
263
264         Graphics2D g2d = (Graphics2D) g;
265         g2d.setRenderingHint(
266             RenderingHints.KEY_ANTIALIASING,
267             RenderingHints.VALUE_ANTIALIAS_ON);
268     }

```

```

268     float[] dashPattern = {4f, 4f};
269
270     Stroke dashedStroke = new BasicStroke(
271         1.5f,
272         BasicStroke.CAP_BUTT,
273         BasicStroke.JOIN_MITER,
274         10.0f,
275         dashPattern,
276         0.0f);
277
278     Stroke solidRedStroke = new BasicStroke(3.0f);
279
280     g2d.setFont(new Font("SansSerif", Font.PLAIN, 13));
281     FontMetrics fm = g2d.getFontMetrics();
282
283     // GABAR GAKLA ANTAR node
284     for (String node : graph_8002.keySet()) {
285         Point p1 = positions.get(node);
286         int w1 = fm.stringWidth(node);
287
288         for (String neighbor : graph_8002.get(node)) {
289             Point p2 = positions.get(neighbor);
290             int w2 = fm.stringWidth(neighbor);
291
292             int x1 = p1.x + w1 / 2;
293             int y1 = p1.y;
294             int x2 = p2.x + w2 / 2;
295             int y2 = p2.y;
296
297             if (p1.y == p2.y) {
298                 if (p1.x < p2.x) {
299                     x1 = p1.x + w1 + 5;
300                     x2 = p2.x - 5;
301                 } else {
302                     x1 = p1.x - 5;
303                     x2 = p2.x + w2 + 5;
304                 }
305             } else if (p1.x == p2.x) {
306                 if (p1.y < p2.y) {
307                     y1 = p1.y + 5;
308                     y2 = p2.y - fm.getAscent();
309                 } else {
310                     y1 = p1.y - fm.getAscent();
311                     y2 = p2.y + 5;
312                 }
313             }
314
315             boolean isPathEdge = false;
316
317             if (finalPath.isEmpty())

```

```

322         && finalPath.contains(node)
323         && finalPath.contains(neighbor)) {
324
325         int idx1 = finalPath.indexOf(node);
326         int idx2 = finalPath.indexOf(neighbor);
327
328         if (Math.abs(idx1 - idx2) == 1) {
329             isPathEdge = true;
330         }
331     }
332
333     if (isPathEdge) {
334         g2d.setColor(Color.RED);
335         g2d.setStroke(solidRedStroke);
336     } else {
337         g2d.setColor(Color.DARK_GRAY);
338         g2d.setStroke(dashedStroke);
339     }
340
341     g2d.drawLine(x1, y1, x2, y2);
342 }
343 }
344
345 // Sambac nama node
346 for (String node : graph_8002.keySet()) {
347
348     Point p = positions.get(node);
349
350     if (!finalPath.isEmpty() && finalPath.contains(node)) {
351         g2d.setColor(Color.RED);
352         g2d.setFont(new Font("SansSerif", Font.BOLD, 13));
353     } else if (visitedNodes.contains(node)) {
354         g2d.setColor(new Color(0, 128, 255));
355         g2d.setFont(new Font("SansSerif", Font.BOLD, 13));
356     } else {
357         g2d.setColor(Color.BLACK);
358         g2d.setFont(new Font("SansSerif", Font.PLAIN, 13));
359     }
360
361     g2d.drawString(node, p.x, p.y);
362 }
363 }
364 }
365 }
366 }
367
368 public static void main(String[] args) {
369     SwingUtilities.invokeLater(() -> {
370         new PetaKampus_2511538002().setVisible(true);
371     });
372 }
373 }
374 }
375 }

```

2. penjelasan code programnin dari tugas tersebut:

1. Deklarasi Package dan Import

```

package pekan9_2511538002;

import javax.swing.*;
import java.awt.*;
import java.util.*;
import java.util.List;

```

Package digunakan untuk menentukan Lokasi file java dalam project, java.swing diguanakn untuk membbuat tampilan GUI (Jframe,Jbutton, Jcomobox, JTextArea , dll, java.awt.* digunakan untuk pengaturan layout, warna, font, dan gambar graf. Java.util.* digunakan untuk struktur data seperti map, list, queue stack, dan set.

2. Deklarasi class

```
public class PetaKampus_2511538002 extends JFrame
```

Klasse ini diutamakan yang mewarisi (extends) JFrame sehingga dapat sebuah jendela aplikasi GUI.

3. Variable

```
private Map<String, List<String>> graph_8002 =  
new LinkedHashMap<>();  
private Map<String, Point> positions = new  
HashMap<>();
```

Syntax ini digunakan untuk menyimpan struktur graf kampus, dan menyimpan hubungan antar Lokasi seperti perpustakaan terhubung ke fakultas dan rektorat dan menyimpan posisi node pada panel graf.

4. Hasil pencarian

```
private List<String> visitedNodes = new ArrayList<>();  
private List<String> finalPath = new ArrayList<>();  
private int exploredCount = 0;
```

visitedNodes untuk menyimpan semua node yang sudah pernah dikunjungi. finalPath menyimpan jalur akhir dari node awal ke node tujuan dan exploredCount untuk menghitung jumlah node yang dikunjungi.

5. Komponen Gui

```
private JComboBox<String> startCombo;  
private JComboBox<String> goalCombo;  
private JTextArea resultArea;  
private GraphPanel graphPanel;
```

Dari syntax ini punya fungsinya masing masing seperti startCombo untuk memilih Lokasi awal, goalCombo untuk memilih Lokasi tujuan, resultArea untuk menampilkan hasil pencarian, dan untuk graf kampus

6. Constructor

```
public PetaKampus_2511538002()
setTitle(...)
setSize(...)
setDefaultCloseOperation(...)
initGraphData();
JPanel topPanel
graphPanel = new GraphPanel();
resultArea
btnBFS.addActionListener(e -> BFS());
btnDFS.addActionListener(e -> DFS());
btnReset.addActionListener(e -> resetGraph());
```

Constructor ini berfungsi untuk menginisialisasi seluruh komponen program saat aplikasi dijalankan. Di dalamnya terdapat pengaturan jendela aplikasi menggunakan `setTitle()`, `setSize()`, dan `setDefaultCloseOperation()`, pembuatan data graf melalui `intiGrapData()`, pembuatan panel control (`topanel`), panel visualisasi graf (`grapphpanel`), serta area hasil pencarian (`resultArea`). Selain itu, constructor juga menggunakan tombol BFS, DFS dan RESET dengan method yang sesuai menggunakan `ActionListener`, sehingga pengguna dapat menjalankan pencarian jalur atau mengembalikan tampilan ke kondisi awal.

7. Method `initGraphData()`

```
Private void intiGraphData
String[] nodes = {
    "Perpus",
    "FTI",
    "Rektorat",
```

```
"PKM",  
"Lab Jaringan",  
"Masjid",  
"Labor AI"  
};  
positions.put("Perpus", new Point(180,50));  
addEdge("Perpus","FTI");
```

Syntax ini digunakan untuk membuat node dan relasi graf, membuat node terdapat 7 lokasi kampus dan menentukan koordinat tampilan, dan yang terakhir membuat hubungan antar node.

8. Method addEdge()

```
private void addEdge(String source, String dest)  
graph_8002.get(source).add(dest);  
graph_8002.get(dest).add(source);
```

Syntax ini digunakan untuk menambahkan sisi (edge) ke graf, karena graf tidak berarah (undirected graf), hubungan dibuat dua arah

9. Method BFS

```
Public void BFS()  
Queue<String> queue
```

Syntax dari query ini digunakan untuk mencari jalur menggunakan algoritma Breadth First Search. Cara kerjanya ambil node awal dan tujuan menggunakan Queue dan kunjungi node per level terus simpan parent node jika tujuan ditemukan, bangun jalur akhir. Kelebihannya itu menjamin jalur terpendek pada graf tidak berbobot.

10. Method DFS()

```
public void DFS()  
Stack<String> stack
```

Perintah dari query ini digunakan untuk mencari jalur menggunakan algoritma Depth First Search, menggunakan Stack untuk menelusuri cabang terdalam terlebih dahulu ; jika buntu, kembali ke node sebelumnya.

11. Method displayPath()

```
public void displayPath(String methodType)  
Hasil Pencarian :  
Jalur :  
Rektorat -> Masjid -> Labor AI  
Node Dikunjungi :  
Rektorat, Perpus, PKM, Masjid, Labor AI  
Jumlah Node Dikunjungi : 5
```

Perintah dari sintaksis ini untuk menampilkan hasil pencarian ke textArea.

12. Method resetGraph()

```
public void resetGraph()  
visitedNodes  
finalPath  
exploredCount
```

Syntax ini digunakan untuk menghapus hasil pencarian sebelumnya, lalu graf dikembalikan ke kondisi awal.

13. Class GraphPanel

```
private class GraphPanel extends JPanel
```

Panel khusus untuk menggambar graf kampus

14. Method paintComponent()

```
protected void paintComponent(Graphics g)  
repaint();
```

```
g2d.drawLine(...)  
Perpus ----- FTI  
g2d.drawString(node, p.x, p.y);  
finalPath.contains(node)  
visitedNodes.contains(node)
```

Menggambarkan seluruh graf secara visual, method ini otomatis dipanggil ketika repaint() dijalankan, menggambar garis yang menghubungkan node-node yang memiliki relasi, dan menggambar jalur hasil jika edge termasuk jalur hasil pencarian, maka diberi warna merah. Menggambar nama node g2d.drawString(node, p.x, p.y); menampilkan nama lokasi kampus di mana node berada pada jalur akhir, merah, biru, dan hitam.

15. Method Main

```
public static void main(String[] args)  
SwingUtilities.invokeLater(() -> {  
    new PetaKampus_2511538002().setVisible(true);  
});
```

Titik awal program dijalankan dan membuat objek JFrame dan menampilkannya.

BAB III

PENUTUP

2.1 kesimpulan

Berdasarkan praktikum yang telah dilakukan, disimpulkan bahwa algoritma Breadth First Search (BFS) dan Depth First Search (DFS) dapat diterapkan untuk menemukan jalur pada graf yang menggambarkan peta lokasi, BFS mencari node berdasarkan kedekatan sehingga cenderung menemukan jalur terpendek pada graf yang tidak memiliki bobot. Sementara itu, DFS menjelajahi hingga kedalaman tertentu sebelum Kembali ke titik sebelumnya untuk menemukan jalur lain. Dalam implementasi program yang dikembangkan, kedua algoritma tersebut mampu mengidentifikasi rute dari titik awal ke titik akhir dan menampilkan urutan node yang dilalui, rute yang diperoleh, dan jumlah node yang dieksplorasi.

DAFTAR PUSTAKA

Introduction to Algorithms. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th Edition). MIT Press.

Data Structures and Algorithms in Java. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.

Fundamentals of Data Structures in C. Horowitz, E., Sahni, S., & Anderson-Freed, S. (2008). *Fundamentals of Data Structures in C*. Universities Press.

Data Structures and Program Design in Java. Kruse, R. L., & Ryba, A. J. (2004). *Data Structures and Program Design in Java*. Pearson Education.